

Data Mining Practice 11

Team 2

양승렬 2200409

최서현 22100133

문벼리 22000245

Task1

01 | 데이터 설명

첫 번째 방법 | glm 사용

02 | 데이터 전처리

03 | 모델 생성

두 번째 방법 | 패키지 사용

02 | 데이터 전처리

03 | Baseline 모델 생성

04 | **Feature importance** 뽑기

05 | 모델 생성

06 | 예측 및 평가

Task2

01 | 전처리

02 | **Regularization**

Task3

01 | 전처리

02 | 예측

Task1

- Train.csv 파일을 trainset & validation set 둘로 임의로 나눈 다음 logistic regression 기반의 multiclass 분류기를 만들고 validation set을 사용하여 성능을 측정하여보라

01 | 데이터 설명

- **목표** : 스마트폰 센서 데이터를 사용하여 사람의 활동을 예측
- [7352, 563]

- $y = \text{activity}$

activity 고유값	의미
STANDING	서 있는 상태
SITTING	앉아 있는 상태
LAYING	누워 있는 상태
WALKING	걸고 있는 상태
WALKING_DOWNSTAIRS	계단을 내려가는 상태
WALKING_UPSTAIRS	계단을 올라가는 상태

첫 번째 방법 | glm 사용

02 | 데이터 전처리

[종속 변수 처리]

```
# 종속 변수 탐색
unique(phone$Activity)
colnames(phone)

# Target 변수 변환 (factor 형태로 변환)
phone$Activity <- as.factor(phone$Activity)
```

[Train, Validation 분할]

```
# Train/Validation Split (80:20 비율)
set.seed(2024)
trainIndex <- createDataPartition(phone$Activity, p = 0.8, li

# 독립 변수들만 추출
dataX <- phone %>% select(-Activity, -subject)
dataY <- phone$Activity # 종속 변수

# Train/Validation 데이터 분할
trainIndex <- createDataPartition(dataY, p = 0.8, list = FALS
trainX <- dataX[trainIndex, ]
```

```
trainY <- dataY[trainIndex]
validX <- dataX[-trainIndex, ]
validY <- dataY[-trainIndex]
```

- createDataPartition를 통해 타깃 변수(Activity)의 클래스 비율을 유지하며 train, test 분할

03 | 모델 생성

[Class 개수만큼 모델 만들기]

```
# 클래스별 이진 모델 학습
modelList <- lapply(levels(factor(dataY)), function(class_label) {
  glm(data = data.frame(y = ifelse(trainY == class_label, 1, 0)),
    formula = y ~ .,
    family = binomial(link = 'logit'))
})
```

- dataY : 다중 클래스(label)를 포함하는 벡터
- lapply를 통해 levels(factor(dataY))에 지정된 함수를 적용
- 결과는 각 클래스에 대한 모델 객체 리스트로 반환

[예측 함수 정의]

```
# 예측 함수 정의
predict_class <- function(inputMat) {
  # inputMat이 행렬 또는 데이터프레임인 경우
  if (is.null(dim(inputMat))) { # 벡터인 경우 : 단일 샘플에 대한 예측
    nData <- data.frame(t(inputMat)) # 입력 벡터 행으로 형태 변환
    result <- sapply(modelList, function(model) {
      # 각 클래스별 모델에서 예측 확률값을 계산
      predict(model, newdata = nData, type = 'response')
    })
    return(levels(factor(dataY))[which.max(result)])
  } else { # 행렬인 경우 : 여러 샘플에 대한 예측
    nData <- data.frame(inputMat)
```

```

# supply를 통해 각 case의 클래스별 모델에서 예측 확률값을 계산
result <- sapply(modelList, function(model) {
  predict(model, newdata = nData, type = 'response')
})
# 가장 높은 확률을 가진 클래스를 반환
return(levels(factor(dataY))[apply(result, 1, which.max)])
}
}

```

- input : `inputMat` [vector or matrix(df)]
- output : `dataY`의 클래스 중 하나

[성능 평가]

```

# 검증 데이터 예측 및 성능 평가
valid_pred <- predict_class(validX)

# accuracy 반환
valid_accuracy <- mean(valid_pred == validY)
cat("Validation Accuracy:", valid_accuracy, "\n")

```

- Validation Accuracy: 0.96

두 번째 방법 | 패키지 사용

02 | 데이터 전처리

[Target 변수 변환 → factor type]

```

# Target 변수 변환 (factor 형태로 변환)
phone$Activity <- as.factor(phone$Activity)

```

[train, valid 분할]

```
# Train/Validation Split (80:20 비율)
set.seed(2024)

# Train index 저장
trainIndex <- createDataPartition(phone$Activity, p = 0.8, li

# train, valid 데이터 저장
trainData <- phone[trainIndex, ]
validationData <- phone[-trainIndex, ]
```

03 | Baseline 모델 생성

[Logistic Regression 모델 생성]

```
# Logistic Regression 모델 생성 (모든 변수 사용)
logistic_model <- multinom(Activity ~ ., data = trainData)
```

- 모든 변수를 사용하여 로지스틱 회귀모델을 생성하려고 했으나

```
> logistic_model <- multinom(Activity ~ ., data = trainData)
nnet.default(X, Y, w, mask = mask, size = 0, skip = TRUE, sof
에서 다음과 같은 에러가 발생했습니다: 가중치의 개수 (3384)가 너무 많습니다
```

- 변수가 너무 많아 모델 생성에 실패했다.
- 따라서 변수 선택이 필요하다고 판단했다.

04 | Feature importance 뽑기

[Feature importance 뽑기]

```
# Random Forest를 통해 Feature importance 뽑기
library(randomForest)
```

```
# Random Forest 모델 학습
rf_model <- randomForest(Activity ~ ., data = trainData, impo

# 특성 중요도 확인
importance <- importance(rf_model)
print(importance)
```

```
> print(importance)
```

	LAYING	SITTING	STANDING	WALKING	WALKING_DOWNSTAIRS	WALKING_UPSTAIRS
tBodyAcc.mean...X	1.231839129	0.39646100	2.64188548	3.4437790324	1.72438979	-1.00903657
tBodyAcc.mean...Y	1.860207777	4.03662070	2.05827210	2.3808187023	2.77859047	1.50184328
tBodyAcc.mean...Z	1.443218336	2.72486214	1.50256038	1.6599702791	-0.02412898	1.87381554
tBodyAcc.std...X	2.977784140	2.21127269	3.58323756	4.1618081558	6.05758216	4.76271793
tBodyAcc.std...Y	1.272487707	0.42817247	2.18917587	1.8731595618	1.10750327	4.56324588

- 변수별로 각 class에 대한 중요도가 표시된다
- 이 중 평균적인 중요도가 높은 상위 10개의 feature를 가지고 학습을 진행

```
# 특성별 평균 중요도 계산
importance_df <- as.data.frame(importance)
mean_importance <- rowMeans(importance_df)

# 중요도 높은 순으로 정렬
sorted_importance <- sort(mean_importance, decreasing = TRUE)

# 상위 10개 특성 선택
top_features <- names(sorted_importance[1:10])
```

▼ Feature importance가 높은 변수

변수명	의미
tGravityAcc.min...X	시간 영역에서 X축 중력 가속도의 최소값
tGravityAcc.min...Y	시간 영역에서 Y축 중력 가속도의 최소값
tGravityAcc.max...X	시간 영역에서 X축 중력 가속도의 최대값
tGravityAcc.max...Y	시간 영역에서 Y축 중력 가속도의 최대값
angle.X.gravityMean.	X축과 중력 벡터 평균 간의 각도
angle.Y.gravityMean.	Y축과 중력 벡터 평균 간의 각도
tGravityAcc.mean...X	시간 영역에서 X축 중력 가속도의 평균값

tGravityAcc.mean...Y	시간 영역에서 Y축 중력 가속도의 평균값
tGravityAcc.energy...X	시간 영역에서 X축 중력 가속도의 신호 에너지
tGravityAcc.energy...Y	시간 영역에서 Y축 중력 가속도의 신호 에너지

05 | 모델 생성

[새로운 Logistic Regression 모델 생성]

```
# 상위 10개 특성만으로 모델 재학습
trainData_reduced <- trainData[, c(top_features, "Activity")]
validationData_reduced <- validationData[, c(top_features, "A

# Logistic Regression 재학습
logistic_model_reduced <- multinom(Activity ~ .,
data = trainData_reduced, maxit = 1000)
```

```
> logistic_model_reduced <- multinom(Activity ~ .,
  data = trainData_reduced, maxit = 1000)
# weights: 72 (55 variable)
initial value 10542.712717
iter 10 value 5840.772641
iter 20 value 5094.277816
iter 30 value 4548.523293
iter 40 value 4468.960231
iter 50 value 4428.351139
iter 60 value 4393.049012
iter 70 value 4365.856584
iter 80 value 4346.059829
iter 90 value 4331.017455
iter 100 value 4317.028791
iter 110 value 4299.410960
iter 120 value 4295.537191
iter 130 value 4284.658155
iter 140 value 4270.233153
iter 150 value 4248.296892
iter 160 value 4219.274486
iter 170 value 4206.298073
iter 180 value 4190.690093
```

```

iter 190 value 4179.661244
iter 200 value 4173.891943
iter 210 value 4172.442806
iter 220 value 4170.969708
iter 230 value 4170.159388
iter 240 value 4169.669565
iter 250 value 4168.949793
iter 260 value 4168.257401
iter 270 value 4167.677712
iter 280 value 4166.674895
iter 290 value 4166.133810
iter 300 value 4165.654522
iter 310 value 4165.190086
iter 320 value 4165.102884
iter 330 value 4165.038045
final value 4164.938258
converged

```

- iter 330부근에서 수렴한 것을 확인

06 | 예측 및 평가

[Validation 예측 및 평가]

```

# Validation 예측 및 평가
predictions <- predict(logistic_model_reduced, validationData)
conf_matrix <- confusionMatrix(predictions, validationData_re
print(conf_matrix)

```

Prediction	Reference					
	LAYING	SITTING	STANDING	WALKING	WALKING_DOWNSTAIRS	WALKING_UPSTAIRS
LAYING	280	0	0	0	0	0
SITTING	1	205	30	11	9	8
STANDING	0	46	212	69	3	11
WALKING	0	2	12	118	38	25
WALKING_DOWNSTAIRS	0	4	9	19	125	33
WALKING_UPSTAIRS	0	0	11	28	22	137

- valid accuracy = 0.73

[추가 실험]

- feature 20개 선택

```
# 상위 20개 특성 선택
top_features <- names(sorted_importance[1:20])

# 상위 20개 특성만으로 모델 재학습
trainData_reduced <- trainData[, c(top_features, "Activity")]
validationData_reduced <- validationData[, c(top_features, "Activity")]

# Logistic Regression 재학습
logistic_model_reduced <- multinom(Activity ~ .,
data = trainData_reduced, maxit = 1000)

# Validation 예측 및 평가
predictions <- predict(logistic_model_reduced, validationData_reduced)
conf_matrix <- confusionMatrix(predictions, validationData_reduced$Activity)
print(conf_matrix)
```

Prediction	Reference					
	LAYING	SITTING	STANDING	WALKING	WALKING_DOWNSTAIRS	WALKING_UPSTAIRS
LAYING	281	0	0	0	0	0
SITTING	0	219	37	0	0	0
STANDING	0	38	237	0	0	0
WALKING	0	0	0	230	5	5
WALKING_DOWNSTAIRS	0	0	0	6	186	8
WALKING_UPSTAIRS	0	0	0	9	6	201

- valid accuracy = 0.92

[test 예측]

```
# 1. test 데이터 불러오기
test<- read.csv("test.csv")
test$Activity

# 2. 테스트 데이터에서 중요 특성만 선택
```

```

test_reduced <- test[, c(top_features, "Activity")] # 중요 특

# 3. Activity를 factor로 변환
test_reduced$Activity <- as.factor(test_reduced$Activity)

# 4. 예측 수행
test_predictions <- predict(logistic_model_reduced, test_reduced)

# 5. 성능 평가 (Confusion Matrix)
test_conf_matrix <- confusionMatrix(test_predictions, test_reduced$Activity)
print(test_conf_matrix)

# 6. 정확도 출력
cat("Test Accuracy:", test_conf_matrix$overall["Accuracy"], "\n")

```

- test accuracy = 0.88

Task2

- Optimize the model by performing feature engineering, such as adding new variables or removing unnecessary ones. This optimization process should only use the training set and validation set, not the Test.csv file.

입력 데이터

파라미터	설명
x	독립 변수(피처) 행렬 (numeric matrix, 반드시 행렬 형태여야 함).
y	종속 변수(타겟 변수). 범주형(factor) 또는 수치형 벡터.
weights	각 샘플에 대한 가중치. 기본값은 NULL (모든 샘플 동일 가중치).

모델 종류

파라미터	설명
family	모델의 분포 유형:
	- "gaussian" : 선형 회귀 (기본값)

	- <code>"binomial"</code> : 이진 분류
	- <code>"multinomial"</code> : 다중 클래스 분류
	- <code>"poisson"</code> : 포아송 회귀 (카운트 데이터)
	- <code>"cox"</code> : 생존 분석 (Cox 회귀)
alpha	Regularization 혼합 비율:
	- <code>alpha = 1</code> : Lasso Regularization
	- <code>alpha = 0</code> : Ridge Regularization
	- <code>0 < alpha < 1</code> : Elastic Net Regularization

교차 검증 관련

파라미터	설명
<code>nfolds</code>	교차 검증에서의 fold 수. 기본값: <code>10</code>
<code>foldid</code>	각 샘플에 대해 fold를 지정하는 인덱스 벡터 (수동으로 fold를 지정할 때 사용)
<code>grouped</code>	- <code>TRUE</code> (기본값): fold를 그룹화하여 평균 손실을 계산 - <code>FALSE</code> : 각 fold에서 개별 관측치를 기반으로 손실 계산

성능 측정 방식

파라미터	설명
<code>type.measure</code>	교차 검증에서 성능을 측정하는 방식 (분석 유형에 따라 자동 설정):
	- <code>"mse"</code> : 평균 제곱 오차 (Mean Squared Error) (기본값)
	- <code>"mae"</code> : 평균 절대 오차 (Mean Absolute Error)
	- <code>"class"</code> : 분류 오류율 (분류 문제)
	- <code>"auc"</code> : ROC 곡선 아래 면적 (이진 분류)
	- <code>"deviance"</code> : 편차 (Deviance)
	- <code>"lambda"</code> : 특정 <code>lambda</code> 값에서 손실 측정

01 | 전처리

[이전과 동일한 전처리]

```
# Train/Validation Split (80:20 비율)
set.seed(2024) # 결과 재현성을 위해 시드 설정
trainIndex <- createDataPartition(train$Activity, p = 0.8, li
trainData <- train[trainIndex, ]
validationData <- train[-trainIndex, ]

# 독립 변수(x)와 종속 변수(y) 분리
x_train <- as.matrix(trainData[, -which(names(trainData) == "
y_train <- as.factor(trainData$Activity)

x_valid <- as.matrix(validationData[, -which(names(validation
y_valid <- as.factor(validationData$Activity)
```

02 | Regularization

1. Regularization

- 정규화란, 가중치 값의 크기를 제한하여 모델의 복잡도를 줄이고 과적합을 방지하는 과정이다.

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)}) + \frac{\lambda}{2m} \boxed{\text{○○○}}$$

→ 이 자리에 어떤 게 들어가냐에 따라 종류가 달라진다.
Regularization parameter : 여가 클수록 가중치를 더 많이 제한한다.

1. Lasso 정규화

① L1 (Lasso)

$$||w||_1 = \sum_{j=1}^{n_x} |w_j|$$

→ 모델 압축을 위해 주로 사용한다.

→ L1 정규화는 가중치의 절대값에 대한 합을 손실함수에 추가하여 이루어진다.

→ L1 정규화는 가중치를 0으로 만들기 위한 경향이 있다. → 일부 가중치가 0이 되면서 모델에 불필요한 특성 제거

→ 간단 모델, 해석 용이, 계산비용 감소, 일반화 능력 향상

```
# L1 Regularization (Lasso) 적용
lasso_model <- cv.glmnet(x_train, y_train, family = "multinom
alpha = 1, nfolds = 10)
plot(lasso_model)
```

```

# 최적의 lambda 값 출력
best_lambda <- lasso_model$lambda.min
cat("Lasso Regularization의 최적의 lambda 값:", best_lambda, "\n")
# 0.0002044492

# 검증 데이터 예측
predictions_base <- predict(lasso_model, newx = x_valid, type = "response")
predictions_base <- as.factor(predictions_base)

y_valid <- as.factor(y_valid)

# 성능 평가
conf_matrix_lasso_base <- confusionMatrix(predictions_base, y_valid)
print(conf_matrix_lasso_base)

```

- valid accuracy = 0.98

2. Ridge 정규화

② L2 (Ridge)

$$||w||_2^2 = \sum_{j=1}^n w_j^2 = w^T w$$

- L2 정규화는 가중치의 제곱에 대한 합을 손실함수에 추가하여 이루어진다.
- L1와는 다르게 가중치가 0에 가까워지지만 0이 되지는 않는다.
- L2 정규화는 모델의 특성을 보다 부드럽게 변화시키는 경향이 있다.

```

# L2 Regularization (Ridge) 적용
ridge_model <- cv.glmnet(x_train, y_train,
family = "multinomial", alpha = 0, nfolds = 10)

# 최적의 lambda 값 출력
best_lambda_ridge <- ridge_model$lambda.min
cat("Ridge Regularization의 최적의 lambda 값:", best_lambda_ridge, "\n")

```

```
# 검증 데이터 예측
predictions_ridge <- predict(ridge_model, newx = x_valid, type = "raw")
predictions_ridge <- as.factor(predictions_ridge)

# 성능 평가
conf_matrix_ridge <- confusionMatrix(predictions_ridge, y_valid)
print(conf_matrix_ridge)
```

- valid accuracy = 0.97

Task3

- Using the final model created in Task 2, measure its performance on the Test.csv file.01

01 | 전처리

[test data 예측을 위해 matrix]

```
# 테스트 데이터 준비
x_test <- as.matrix(test[, -which(names(test) == "Activity")])
y_test <- as.factor(test$Activity)
```

02 | 예측

[Lasso base 모델로 예측]

- 성능이 가장 좋았던 Lasso base 모델로 예측

```
# 기본 Lasso 모델로 테스트 데이터 예측
predictions_test_lasso <- predict(lasso_final_model, newx = x_test, type = "raw")
predictions_test_lasso <- as.factor(predictions_test_lasso)

# 기본 Lasso 모델의 테스트 성능 평가
conf_matrix_test_lasso <- confusionMatrix(predictions_test_lasso, y_test)
```

```
cat("\n기본 Lasso Regularization 성능 평가 (테스트 세트):\n")
print(conf_matrix_test_lasso)
```

- test accuracy = 0.945

모델	Accuracy
LASSO_BASE	0.945
RIDGE_BASE	0.9393